GitHub-Anleitung StartKlar

Nov 11, 2026 · @Chupa

Überblick

Unser gesamter Code liegt im GitHub-Repository Nzchupa/StartKlar. Jede/r arbeitet auf einer lokalen Kopie am Laptop und lädt Änderungen über Git wieder hoch.

Die wichtigsten Bereiche oben in der Leiste:

Reiter	Wofür wir ihn nutzen
Code	Dateien ansehen, Branch wählen (`main`), Repository-URL zum Klonen holen (grüner Button **Code**)
Issues	Unsere Aufgaben, z. B. #1 „Funktionale Anforderungen sammeln“
Pull requests	Änderungen aus einem Branch prüfen und in `main` übernehmen
Branches (über „1 Branch“)	Alle Arbeitszweige; `main` ist der Standard-Branch

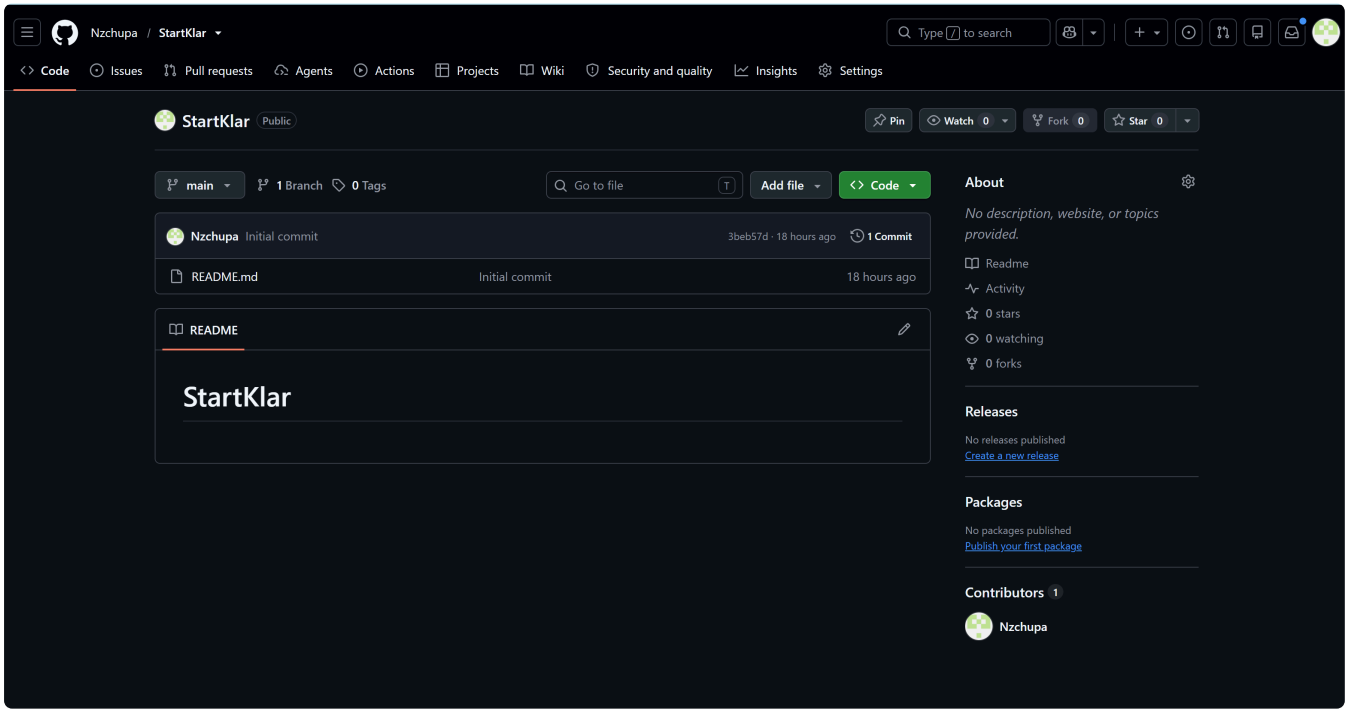


Abbildung 1: Startseite des Repositorys StartKlar

Voraussetzungen

Vor dem ersten Klonen braucht jede/r vier Dinge:

1. **GitHub-Account** und Zugriff aufs Repository: Nzchupa lädt euch unter *Settings* → *Collaborators* ein, ihr bestätigt die Einladung per E-Mail. Ohne Einladung könnt ihr klonen, aber nicht pushen.
2. **Git für Windows** von git-scm.com installieren. Dabei wird auch **Git Bash** installiert, das Terminal aus den Screenshots.
3. **Name und E-Mail** einmalig in Git Bash eintragen (gleiche E-Mail wie bei GitHub):

```
git config --global user.name "Vorname Nachname"
git config --global user.email "eure@mail.de"
```

4. **Eclipse IDE for Java Developers** – das Git-Plugin (EGit) ist dort schon enthalten.

Beim ersten `git push` öffnet sich ein Browserfenster zur GitHub-Anmeldung. Danach merkt sich Git die Zugangsdaten.

Repository auf den Laptop klonen

Klonen macht ihr nur einmal: danach liegt ein Ordner `StartKlar` mit dem ganzen Projekt auf eurem Laptop.

1. github.com/Nzchupa/StartKlar öffnen.
2. Auf den grünen Button **Code** klicken → Reiter **Local** → **HTTPS**.
3. Die URL `https://github.com/Nzchupa/StartKlar.git` mit dem Kopier-Symbol daneben kopieren.

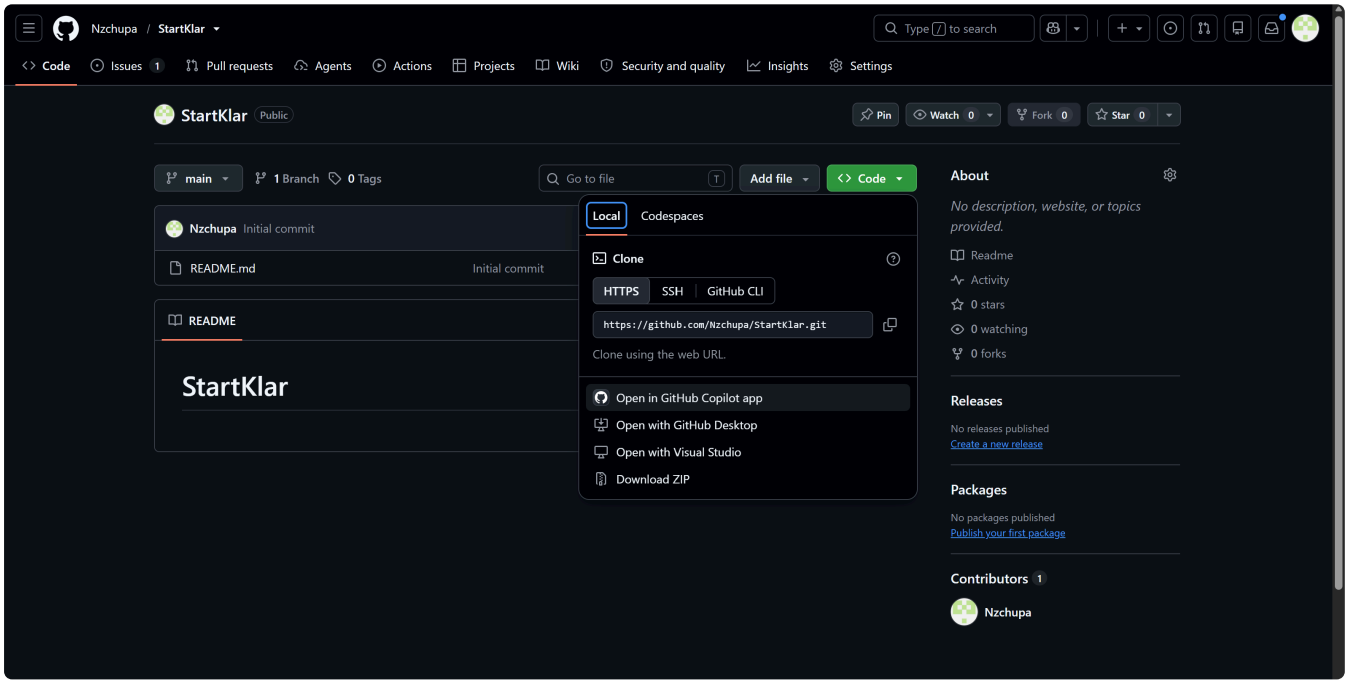


Abbildung 2: Code-Button mit der HTTPS-URL zum Klonen

4. Git Bash öffnen und in den Ordner wechseln, in dem das Projekt liegen soll, z. B. euren Eclipse-Workspace:

```
cd "C:\Users\<Name>\OneDrive\Uni\Info_I"
```

5. Klonen und in den neuen Ordner wechseln (Einfügen in Git Bash: Rechtsklick → Paste oder Umschalt+Einfg):

```
git clone https://github.com/Nzchupa/StartKlar.git
cd StartKlar
```

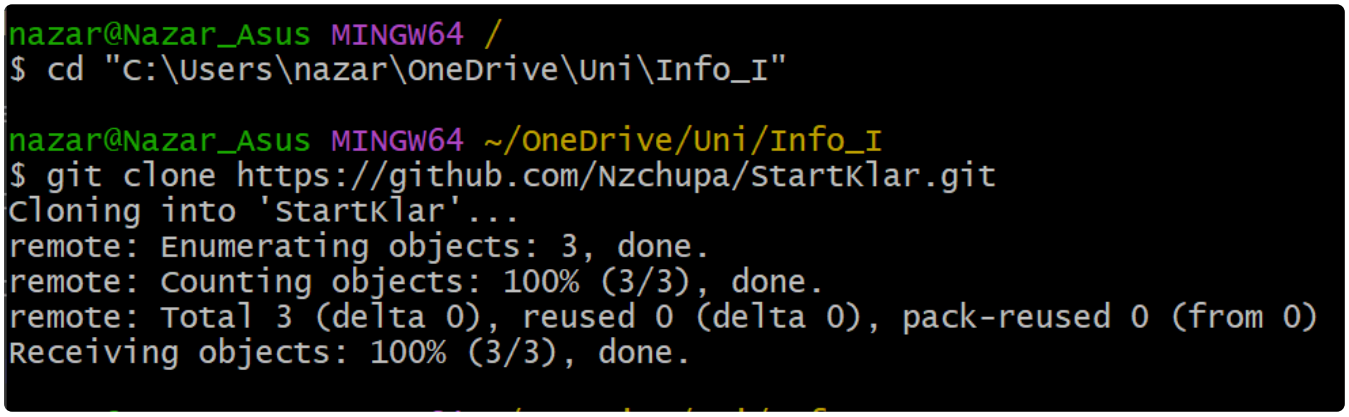


Abbildung 3: git clone in Git Bash

Windows-Pfade in Anführungszeichen setzen, dann funktionieren Backslashes und Leerzeichen. Die Zeile `Receiving objects: 100% … done` zeigt, dass das Klonen geklappt hat.

Täglicher Arbeitsablauf

Jede Aufgabe läuft gleich ab: neuesten Stand holen, eigenen Branch anlegen, committen, pushen, Pull Request öffnen. Alle Befehle laufen in Git Bash im Ordner `StartKlar`.

1. **Neuesten Stand holen** – immer vor Arbeitsbeginn:

```
git checkout main
git pull
```

2. **Eigenen Branch anlegen** – Name = Issue-Nummer + Stichwort:

```
git checkout -b 1-anforderungen
```

3. **Arbeiten** – Dateien in Eclipse oder einem Editor ändern und speichern.
4. **Änderungen prüfen und committen:**

```
git status
git add .
git commit -m "Funktionale Anforderungen ergänzt"
```

5. **Hochladen** – beim ersten Push des Branches mit `-u`, danach reicht `git push`:

```
git push -u origin 1-anforderungen
```

6. **Pull Request öffnen** – auf GitHub erscheint der gelbe Hinweis **Compare & pull request**. Klicken, kurz beschreiben, in die Beschreibung `Closes #1` schreiben (schließt das Issue beim Mergen automatisch), jemanden als Reviewer eintragen.
7. **Mergen** – nach dem Review klickt jemand **Merge pull request**. Danach lokal wieder bei Schritt 1 anfangen.

Issues und Branches

Im Reiter **Issues** stehen unsere Aufgaben. Jede/r sucht sich ein Issue, trägt sich rechts unter **Assignees** ein und arbeitet dafür in einem eigenen Branch.

- **Neues Issue:** grüner Button **New issue**, Titel + kurze Beschreibung.
- **Eigene Aufgaben finden:** links **Assigned to me**.
- **Erledigt:** das Issue schließt sich automatisch, wenn der Pull Request mit `Closes #<Nummer>` gemergt wird.

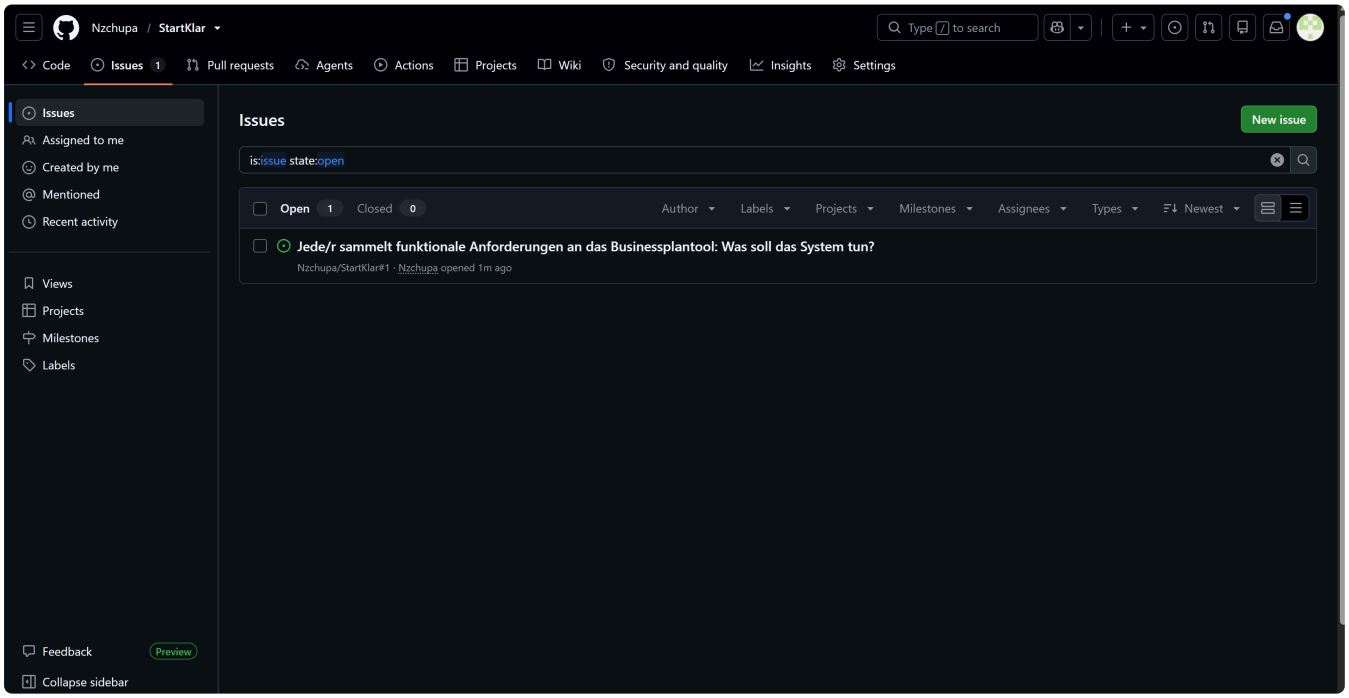


Abbildung 4: Issue-Übersicht mit Issue #1

Unter **Code → 1 Branch** seht ihr alle Branches. `main` ist der Standard-Branch (*Default*) und enthält nur geprüften Stand. Einen Branch könnt ihr auch hier über **New branch** anlegen und danach lokal mit `git fetch` und `git checkout <name>` holen.

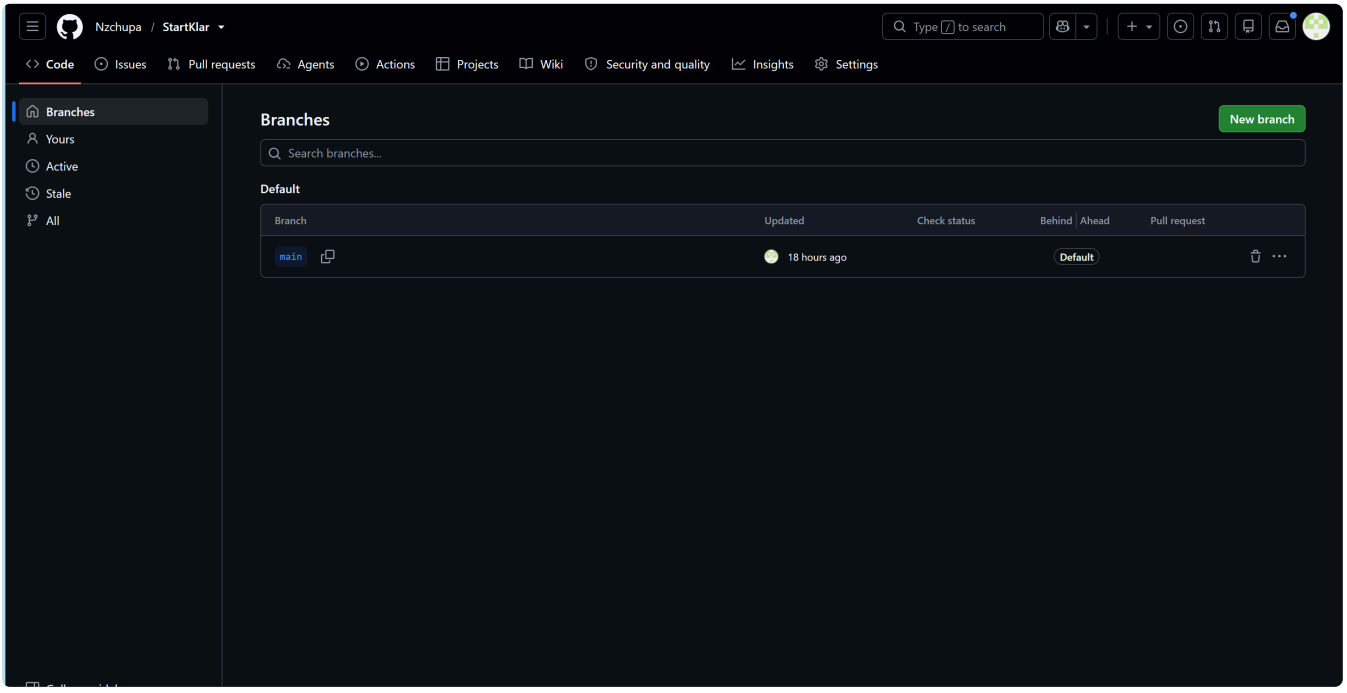


Abbildung 5: Branch-Übersicht mit dem Standard-Branch main

Projekt in Eclipse öffnen

Wenn ihr schon mit Git Bash geklont habt, importiert ihr den Ordner einfach in Eclipse:

1. **File → Import… → Git → Projects from Git (with smart import)** → Next.
2. **Existing local repository** → **Add…** → den Ordner `StartKlar` auswählen → Next.
3. Eclipse erkennt das Projekt automatisch → **Finish**.

Ohne vorheriges Klonen geht es direkt in Eclipse: im 2. Schritt **Clone URI** wählen, die HTTPS-URL einfügen, als User euren GitHub-Namen und als Passwort einen Personal Access Token eintragen (GitHub akzeptiert hier kein normales Passwort), Branch `main` wählen.

Git direkt aus Eclipse nutzen:

- **Pull:** Rechtsklick aufs Projekt → **Team → Pull**.
- **Commit + Push: Window → Show View → Other… → Git → Git Staging** öffnen, Dateien nach *Staged Changes* ziehen, Nachricht schreiben, **Commit and Push**.
- **Branch wechseln/anlegen:** Rechtsklick → **Team → Switch To → New Branch…**

Wichtig: Den Ordner `bin/` und die Eclipse-Metadaten (`.metadata`) nicht committen – dafür legen wir eine `.gitignore` an.

Teamregeln und häufige Fehler

Regeln:

- Nie direkt auf `main` committen – immer über Branch + Pull Request.
- Vor jeder Arbeitssitzung `git pull` auf `main`.
- Kleine Commits mit aussagekräftiger Nachricht („Login-Maske ergänzt“ statt „Änderungen“).
- Ein Branch pro Issue; nach dem Mergen kann der Branch gelöscht werden.

Meldung	Ursache	Lösung
`gti: command not found`	Tippfehler	`git` schreiben
`not a git repository`	Falscher Ordner	`cd StartKlar`
`Permission denied` / `403`	Keine Einladung oder falsche Anmeldung	Einladung annehmen; in Eclipse Token statt Passwort
`rejected … fetch first`	Remote ist neuer als euer Stand	`git pull`, dann erneut `git push`
`CONFLICT (content)`	Zwei Personen haben dieselbe Stelle geändert	Datei öffnen, Bereiche zwischen `<<<<<<<` und `>>>>>>>` bereinigen, `git add .`, `git commit`